

SECTION 11: Safety, Rate Limiting & Respectful Scraping

AGENDA:

Today we are going to understand how we can use AI agents to interact with real websites and browsers

We will start at 9:07 pm

Section 1 :Why Agents needs browser

Section 2: Playwright Basics for Agent Integration

What is Playwright?

Playwright is a browser automation library developed by Microsoft.

It allows us to programmatically control modern web browsers using Python.

Playwright supports:

Chromium
Firefox
WebKit

For AI agents, Playwright can act as the browser-control layer.

The AI agent decides what needs to be done, while Playwright performs the actual browser interaction.

Important Features of Playwright:

1. Auto-waiting - waits for elements to be actionable before acting
2. Network interception out of the box
3. Multiple browser contexts in one process - cheap parallelism

SECTION 3 : Priority order (most robust ? most fragile):

Rule for agents: prefer role and text locators - they match how a human (and an LLM reading the accessibility tree) describes the page.

Section 4:Auto-Wait - Why Playwright is Agent-Friendly

When you call `click`, Playwright first checks:

1. Element is attached to the DOM
2. Element is visible
3. Element is stable (not animating)
4. Element receives events (not covered by an overlay)
5. Element is enabled

Only when all five pass does the click fire - retrying up to the timeout (default 30s).

For an LLM agent, this is huge. The model doesn't need to reason about page load timing.

Explicit waits (when you need more control):

`waitForFunction` is overused. Sites like Flipkart/Amazon never truly go idle due to background pings. Prefer waiting on a specific element or URL pattern.

SECTION 5 : Dynamic Content and SPAs

Most modern Indian consumer sites (Flipkart, Swiggy, Zomato, Cred) are Single Page Applications. They load a near-empty HTML shell, then JavaScript builds the page. `document.body.innerHTML` returns almost nothing.

Playwright runs a real browser, so JavaScript executes normally. But you now need to know when the page is done.

Three patterns:

1. Wait for a specific element you actually need ? (gold standard)
2. Intercept the underlying network/API call for clean JSON
3. Infinite scroll - scroll, wait for new items, repeat (always with a cap!)

SECTION 6: Authentication Pattern 1: `storage\_state` (Production Pattern)

The problem: Logging in from scratch every run is slow, triggers anti-bot heuristics, and may trigger 2FA.

The solution: Log in once ? save cookies + localStorage to a JSON file ? every subsequent run loads that file.

Analogy: The Remember me on this device checkbox. You authenticate once; your machine becomes a trusted device. is the agents trusted-device certificate.

On Colab (no interactive display), the flow is:

1. Run an interactive script locally (), log in by hand, save the state file
2. Upload that file to Colab
3. Load the state file in Colab

?? Security: contains session cookies. Anyone with this file can act as you. Treat it like an SSH key - never commit it, add to , store encrypted in production (AWS Secrets Manager, Vault, etc.)

Section 7: Authentication Pattern 2: Programmatic Login

Use when: rotating accounts, service accounts with credentials in a secret manager, or fresh onboarding flows.

?? Demo below uses publicly published credentials from a practice site (). In real code, pull from Colab Secrets - never hardcode passwords.

BREAK

Section 8: browser-use Library Deep Dive

What is browser-use and why does it exist?

Playwright gives you eyes and hands. But theres a gap between and my LLM decides on its own what to click on a page it has never seen.

browser-use fills that gap. Its an open-source Python library that sits on top of Playwright and gives your LLM a structured way to perceive and act on web pages.

Analogy: Playwright is the steering wheel and pedals. browser-use is the chauffeur who can read road signs. You give instructions in natural language; the chauffeur translates them into precise mechanical

actions.

What browser-use does for you:

1. Perception - serializes the page into a representation the LLM can reason about
2. Action space - fixed vocabulary: click element 7, type into element 12, scroll, go back
3. Loop - perceive ? think ? act, including retries and self-correction
4. Extensibility - custom actions, any LangChain-compatible LLM, Pydantic output

How the LLM "sees" a page

Mode 1: DOM serialization with element indexing (default)

browser-use walks the DOM, finds all interactive elements, filters invisible ones, and produces a numbered list:

The LLM sees this, decides click element 1, emits . Fast, token-efficient, robust.

Mode 2: Vision (screenshot)

browser-use takes a screenshot, draws colored bounding boxes with index numbers, and sends it to a vision-capable LLM. More reliable for visually complex pages (Canva, Figma) but costs more tokens.

For text-only Groq models like , we use DOM-only mode ().

Section 9: Intermediate: Structured Output with Pydantic

Section 10: Controlling the Loop: System Prompts and Policies

The `system_prompt` parameter is where you encode your agents policies - the equivalent of writing an SOP for your chauffeur. (In browser-use 0.1.40 this is the supported hook for injecting extra instructions; later versions add `extra_instructions`.)

SECTION 11: Safety, Rate Limiting & Respectful Scraping

The Honest Framing on Legality and Ethics

Web scraping occupies a legal grey zone that varies by country, site, and use case.

Three layers to understand:

1. - voluntary file at the root of every site. Not legally binding in most jurisdictions, but ignoring it is a clear signal of bad faith.
2. Terms of Service - most sites prohibit automated access. The hiQ v LinkedIn case (2022 US) established that scraping public data is generally not a CFAA violation, but login-gated content is messier.
3. Data protection laws - GDPR (EU), DPDP Act (India, 2023), CCPA (California). Personal data = regulated territory.

The lawyer test: If you'd be embarrassed to explain your scraper to the site's lawyer, don't run it. This single test catches 95% of bad decisions.

Three things you should always do:

1. Identify yourself - set a user-agent with a contact email:
2. Respect unless you have explicit permission
3. Avoid login-gated content unless you are the account owner